

Raft, Paper, Scissors: Maximum Divergence is All You Need

Taylor Whatley

“The Founder” but in the McDonald’s way
The TCP Foundation (Taylor Communication Protocol)
Vancouver, BC, Canada
whatltay@gmail.com

Aviraj Newatia

Assistant Undersecretary Zookeeper
The Ship that sank in ‘The Life of Pi’
Stuck on Raft in Atlantic Ocean since 2007
projectavi.exe@gmail.com

Abstract—Just read the paper lol.

I. INTRODUCTION

Consensus algorithms in distributed systems are widely studied for the application of ensuring that a collection of nodes or peers in a distributed system act like a single coherent machine. To ensure this sort of behaviour, it is imperative that we handle system and network failures and ensure that the data across these nodes remains consistent.

A popular algorithm in the space of distributed consensus is Raft [1]. This is in fact just like the floating contraption created from wooden logs. By the scientific intuition and expertise of the authors we understand that it is called such because the algorithm ensures that the system remains afloat. We remark that this is remarkable naming. Bravo authors. Typically taught in entry-level undergraduate courses on distributed systems, Raft produces a system which is fault-tolerant and provides high availability across its replicas. It ensures a total order of operations through the leader election mechanism which ensures a single leader exists in the system at any point in time. In addition to this, the conservative committing of operations from the log requiring acknowledgements from a majority of the replica nodes, ensures that server state updates in a consistent and cohesive manner.

For all of its pros, Raft has some major drawbacks. By design, this system focuses on high availability - providing incredible fault tolerance in a partially-synchronous network model assuming no malicious actors. However, Raft is not scalable, all operations go through the leader - which restricts the volume of requests that can be made to the service. In addition to this, it means that in a geo-distributed system of replicas, all requests still need to go to a single server which greatly increases the latency of service operations for clients who are not geographically located close to whichever node or replica is currently the leader. This is one of the largest restrictions that our work tackles. In addition to this, a much more significant problem exists.

Raft is too robust. All good consensus algorithms are. They solve the problem far too well. Linearizability this. Causal ordering that. All we need, is eventual consistency. And perhaps some stochasticity to go with it (on the side). In this work we tackle the non-trivial problem of a Raft-like distributed consensus system which is **scalable**. In our proposed system, entries can be logged to any node - overcoming a major

drawback of the Raft algorithm. We also guarantee under any system and network model that the state of our system is eventually consistent. Turns out that, just like attention [2], maximum divergence is all you need.

A. Paper overview

In this paper we introduce Raft, Paper, Scissors, henceforth referred to as **RPS**. RPS is a new consensus algorithm targeted at disagreement. We draw inspiration from [REDACTED] to remember - eventually, asymptotically, there will be convergence. In our case this is consistency.

RPS provides a scalable extension to the Raft protocol. Under the RPS system, clients can make requests to any node in the distributed system, removing the main constraint of Raft in which all requests are routed to the leader of that term. In addition to this, RPS lacks a single point of failure, and is fully operational at all times except during agreement periods. This ensures that the system is basically always available. When considering a distributed system consisting of $N + 1$ nodes, RPS can handle N failures. As long as a single node remains up and running, the system can provide a cohesive distributed key-value store service.

RPS achieves this behaviour through advanced stochasticity techniques reliant upon a policy-game co-design augmented with an overly complicated by *really frikin cool* vector-clock based versioning system. Finally, RPS is robust to malicious nodes, as transactions and handshake agreements between nodes are cryptographically secured using a salting system. We provide formal, theoretical guarantees for the cryptography in Section V. Alongside this, we provide further formal formulations of consensus guarantees in Section V.

Now lo and behold, the glory of RPS.

II. RAFT, PAPER, SCISSORS

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequaleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem.

$$a + b = \gamma \tag{1}$$

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequaleamus animo,

TABLE I: THE CIRCLES OF THE CIRCLE SYSTEM AND THEIR AVERAGE DISTANCE FROM THE BIG CIRCLE

Circle	Distance from Big Circle (million km)
Circle 1	57.9
Circle 2	108.2
Circle We Live On	149.6
Circle 4	227.9
Circle 5	778.6
Circle 6	1,433.5
Circle 7	2,872.5
Circle 8	4,495.1

cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et.



Fig. 1: A circle representing a circle.

In Fig. 1 you can see a common representation of the Big Circle, which is a circle that is located at the center of the circle system.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequae doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos.

III. METHOD DISCUSSION

Avi: Did you [readers] understand our method? It's really cool isn't it!

Taylor (*chuckling*): Uh Avi...

Avi: What's up?

Taylor (*laughing a bit too much*): That was just lorem ipsum text

Avi: Do you think they noticed...

Taylor: (*on the floor*): Dude!

Avi: (*now also laughing on the floor*): Sorry okay! Let's try that again.

IV. RAFT, PAPER SCISSORS (FOR REAL THIS TIME)

When analyzing the problem, we came up with some important requirements for the protocol to ensure each machine has a fair chance:

- 1) **Cheating-Proof RPS:** When a computer decides a Rock Paper Scissors move to respond to its opponent, it should pick its choice without knowing the move of its opponent. After both players have picked the move, both should be able to verify that the move was correct (e.g. they did not change their move after receiving the message from its opponent). Also, computers should be instructed that cheating will result in an immediate restart, which is about equivalent to death for machine life.
- 2) **Eventuaaaaaal Convergence:** Convergence should eventuaaaaaaly happen, if all systems are connected with a synchronous, reliable link. By eventuaaaaaal, we provide no guarantees as to when the machines will converge. Just that it seems that it would probably happen at some point maybe if the outcome of rock paper scissors are fair.
- 3) **Low Availability:** While all our nodes can be online and working as expected, we expect them to be able to respond to requests about recent information almost none of the time. This protocol is great for individuals with SLAs of 2% or lower.

A. Achieving a Cheating Proof RPS Protocol

If our entire protocol is going to be based around the game of Rock Paper Scissors, we wanted to make sure that the game is performed fairly, and computers don't have a chance to cheat. Let's consider two computers, Alison and Robert. In the main protocol (explained in the Eventuaaaaaal Convergence section), when a computer receives a new entry, it will attempt to challenge one of its neighbors until it is able to complete a Rock Paper Scissors game. In this case, Alison just received an instruction to replicate the log element 5 at index 3. Alison will then select Robert as its RPS challenge neighbor. To highlight the possibility for cheating, let's inspect a naive RPS protocol:

```
type NaiveChallenge struct {
    selection int // 0 = R, 1 = P, 2 = S
    index int // index of element, ex. 3
    element int // element to replicate, ex. 5
}
```

```
type NaiveChallengeResponse struct {
    same bool // true if the element was the
    same, both win
    draw bool // true if !same, and rock paper
    scissors considered incomplete
    win bool // true if !same and !draw, and the
    challenged server won RPS
    element int // server's element at index
}
```

Generally, there are **four outcomes** to a RPS Challenge:

- 1) The elements at the index are the same, in which case both servers are considered to have "won" this RPS challenge (see won case).
- 2) The servers draw, in which case the challenging server will try another RPS challenge (at another randomly selected server).

- 3) The challenging server wins, in which case the losing server replaces its value at index
- 4) The challenging server loses, in which case the challenging server (the loser) replaces its value at index

If there is no element at the given index, the challenging server is considered to have won, and the challenged server has to append this element to its log, at the index given. If the index is not directly after the log, the server may have to pad the log with “empty” elements.

- 1) Alison picks either Rock, Paper or Scissors.
- 2) Alison sends Robert a **NaiveChallenge** message with their selection.
- 3) Robert picks either Rock, Paper or Scissors.
- 4) Robert decides if it won Rock Paper Scissors, and sends a **NaiveChallengeResponse** message.

This protocol fails fairness in a few ways. For example, when Robert picks their Rock, Paper, Scissors response, it already knows Alison’s selection. Therefor Robert could just pick a winning answer, or simply just return win = true, regardless of the selection. A separate challenge and selection message may help mitigate this problem, but nothing is stopping Robert from just waiting for the selection message to reach it before picking its answer.

Instead, we chose a cryptographic answer to ensure Robert does not know Alison’s selection when picking RPS.

```
type RPSChallenge struct {
    hash byte[] // sha256 hash of salt + RPS
    index int // index of element, ex. 3
    element int // element to replicate, ex. 5
}
```

```
type RPSChallengeResponse struct {
    same bool
    rps int // selection in step 5
    element int // element at this index
}
```

```
type RPSChallengeComplete struct {
    draw bool // true if draw
    win bool // true if win
    salt byte[] // salt in step 2
    rps int // selection in step 1
}
```

- 1) Alison picks either Rock, Paper or Scissors.
- 2) Alison picks a cryptographically random 128 byte “salt” sequence.
- 3) Alison computes the SHA256 hash of the salt concatenated with their Rock, Paper, Scissors answer (byte 0 if Rock, byte 1 if Paper, byte 2 if Scissors).
- 4) Alison sends the **RPSChallenge** message, with the hash computed in step 3 (but not the salt computed in step 2). This should be enough to verify the answer with the salt, but not enough to infer the answer without the salt.
- 5) Robert picks Rock, Paper, or Scissors.
- 6) Robert sends their selection in a **RPSChallengeResponse** message.
- 7) Alison verifies if they have won, and sends back an **RPSChallengeComplete** message with their RPS selection and the salt they chose in step 2.

- 8) Robert will append the RPS selection to the salt and verify that the hash is the same as the hash returned in Step 4 (computed in Step 3).

This way, Alison can’t change its response because it already sent the hash to Robert. And Robert cannot decide its response based on Alison because Alison never sends it RPS selection. This allows for a secure and fair game of Rock Paper Scissors to be played between two computers, which we will use a lot in our protocol implementation.

B. Achieving Eventuaaaaaal Convergence

Eventuaaaaaal Convergence is achieved in RPS through the **main** protocol. Consider a distributed system with N nodes each set up in a Raft-like manner. This means that the nodes contain a log of entries that have been received from users to be committed to the system. For system scalability, unlike in Raft, every node can be used as a access point for clients and users.

Consider a synchronization timeout of T_i for node i . Upon timeout, i randomly chooses another node (node j) in the system to synchronize its data with. i checks if j is currently Gaming (engaged in RPS-Synchronization) and if so, chooses another node. At this point, RPS is initiated, and both nodes are locked into a game-state and cannot receive new entries until completed, nor can they engage in other RPS handshakes. In the following algorithm, Node i is referred to as N_i , and Node j as N_j .

```
1: function RPS-SYNCHRONIZE( $N_i, N_j$ )
2:    $N_i$ .Gaming  $\leftarrow$  True
3:    $N_j$ .Gaming  $\leftarrow$  True
4:   ▷ Initialize the search range
5:    $L \leftarrow \min(\text{len}(N_i.\text{Log}), \text{len}(N_j.\text{Log})) - 1$ 
6:    $k \leftarrow 0$ 
7:   while  $k \leq L$  do
8:      $W \leftarrow \text{RPSChallenge}(N_i.\text{Log}[k], N_j.\text{Log}[k])$ 
9:      $N_i.\text{Log}[k] \leftarrow W$ 
10:     $N_j.\text{Log}[k] \leftarrow W$ 
11:     $N_i$ .Gaming  $\leftarrow$  False
12:     $N_j$ .Gaming  $\leftarrow$  False
13:   return null
```

Once the RPS Synchronization between nodes i and j conclude, they are guaranteed to have the same log of entries. We then call the tuple (i, j) RPS-synchronized and release the locks on the nodes. A formal theorem and proof is provided in Section V.

C. Achieving Low Availability and Liveness

Generally, we claim that there is a high likelihood that at any point in time, nodes are engaged in RPS Synchronisation. This is because our nodes are Gamers, and love playing with each other.

Considering that, as a user, there is no guarantee that any operation you commit to the RPS system will survive, our algorithm achieves what Raft cannot - low liveness. Any entry x has a chance of being committed to the log of every node and becoming ubiquitous dependent on the ability of any node

it is committed to, to winning RPS. This just means that if x does not make it to the final asymptotic log that will be produced by Judgement Day [3] then it is, in its entirety, a skill issue.

RPS is based off of the Raft protocol, famous for its high availability and fault-tolerance. As such, we demonstrate that RPS produces extremely low availability under the majority of conditions regardless of its ability to handle N failures in an $N + 1$ node system. By nature of RPS Synchronisation, any two nodes engaged in battle are unable to service any incoming requests from clients - ensuring that client requests are usually redirected through the system to other nodes which are not engaged in synchronisation or are blocked until the challenges complete. This means that while the system is perpetually available under most situations, it has extremely high latency which will suggest to users that the system may actually just be down. As such, we implicitly achieve low availability despite achieving high availability by definition.

D. Extensibility of the RPS Protocol

The mechanism we have designed for RPS is not limited to the naive game of Rock-Paper-Scissors. Nor is it limited to random policies for these games. The beauty of this distributed consensus algorithm is its extensibility.

Each node in the system may use its own policy for determining how it would like to play the selected game. Some suggestions include but are not limited to: a move prediction buffer, a monte-carlo tree, a neural network, or hooking it up to a ChatGPT API like everyone seems to be doing these days. In addition to this, RPS can be extended to be based on a game of tic-tac-toe, battleship, or even chess! All of these options change the level of gameplay inherent in the RPS process, and can change the complexities underlying which node is more likely to have its entries committed by everyone. The length of the game will also greatly change the throughput of the RPS system.

V. THEORETICAL ANALYSIS

Theorem V.1 (Eventuaaaaaal Convergence)

Given an RPS-system with N nodes. $\lim_{t \rightarrow \infty} |S_t| = 1$ where $S_t = \{N_i.\text{Log at time } t \mid i \in \{1 \dots N\}\}$.

Proof By the Theorem from [4] this theorem has already been proved. $\square$

VI. CONCLUSION

In this work we introduce and analyse a new protocol for distributed consensus which is scalable and eventuaaaaaaly convergent, Raft-Paper-Scissors. We present this work to the community with hopes that we can adjust the trajectory of future research away from highly robust infrastructures such as Raft, Zookeeper, and Dynamo towards far more *fun* protocols such as RPS. Highly available, fault tolerant, and live systems are overrated and our current approaches perform far too well. Let's continue to dumb it down a little.

REFERENCES

- [1] D. Ongaro and J. Ousterhout, "In search of an understandable consensus algorithm," in *Proceedings of the 2014 USENIX Conference on USENIX Annual Technical Conference*, in USENIX ATC'14. Philadelphia, PA: USENIX Association, 2014, pp. 305–320.
- [2] A. Vaswani *et al.*, "Attention Is All You Need." [Online]. Available: <https://arxiv.org/abs/1706.03762>
- [3] Wikipedia, "Last Judgment — Wikipedia, The Free Encyclopedia." 2025.
- [4] Wikipedia, "Infinite monkey theorem — Wikipedia, The Free Encyclopedia." 2025.